Logo Evolution Lab — Experiment Edition: Design Document

Core Research Question

Can LLM-driven iterative evolution of programs produce Logo turtle graphics that are significantly better than what the same LLMs can generate in a single shot?

The Logo domain is ideal: programs are short, the language is simple, but the visual output space is rich. If evolution consistently exceeds one-shot generation, that's evidence that LLM-guided search through program space finds regions the model can't reach directly — analogous to how human iterative refinement surpasses first drafts, but with the variation and selection both delegated to AI.

Purpose

A new standalone HTML file (`logo-evolution-lab-experiment.html`) that extends the existing Logo Evolution Lab with:

1. **One-shot baseline generation** — the comparison group
2. **Batch evolutionary runs** — systematic, reproducible
3. **Structural and image metrics** — LLM-independent evaluation
4. **Cross-model critic evaluation** — guarding against single-model bias
5. **Results dashboard** — tables, charts, visual gallery
6. **Structured export** — JSON + CSV for external analysis

The original app remains untouched. This version shares its Logo compiler, renderer, and API layer.

Architecture: What to Copy, What to Add

Copy unchanged from the existing app

- All CSS (variables, card styles, buttons, toggles, tree, modal, etc.)
- `compileLogo()`, `autoFitQueue()`, `renderQueue()`, `drawCode()`, `canvasB64()`
- `callClaude()`, `callGemini()`, `callOpenAI()`, `callAI()` (the full API layer including fallback logic)
- `agentModify()`, `agentCritique()`, `parseCritiqueJSON()`, `cleanCode()`, `validateLogo()`
- `buildPool()`, `addBackups()`, `selectWinner()`, `buildSelectionLog()`
- `magDesc()`, `diffHTML()`, `escHtml()`, `parseMarkdown()`

- `isArtifact()`, `notifyTab()`, theme toggle, modal system
- `MODIFIER_SYS` and `CRITIC_SYS` prompt constants

New components

1. **Experiment Definition Panel** — simplified from the original design
2. **Baseline Generator** — one-shot program generation for comparison
3. **Batch Runner Engine** — runs experiments sequentially
4. **Structural Metrics Analyser** — computes code/image metrics from compile output
5. **Cross-Model Evaluator** — re-judges programs with different models
6. **Raw API Logger** — captures every prompt/response
7. **Results Dashboard** — the key evolution-vs-baseline comparison plus supporting views
8. **Export System** — JSON, CSV, PNG gallery

---

1. Experiment Definition Panel

Simplified to focus on the primary research question. Secondary parameters (magnitude, selection mode, critic mode) use sensible defaults that can be overridden but aren't the main variables.

UI layout

A card titled **"Experiment Setup"** with these sections:

Seed Programs (multi-entry)

A textarea where each line is a separate seed Logo program (or labelled: `spiral: REPEAT 36 [FD 100 RT 170]`). Each seed gets a short ID. These are the starting points for evolutionary runs.

Taste Descriptions (multi-entry)

A list of aesthetic goal strings. Each gets an ID. These define what "better" means for each experimental condition.

Models (multi-select)

Checkboxes for available models: claude-sonnet-4-6, gemini-3-flash-preview, gemini-2.5-pro, gpt-5.4, etc. Each checked model will be tested for both evolution and one-shot generation.

Evaluation Models (multi-select)

Which models should serve as critics when re-evaluating final results. Defaults to all checked models. This produces the model×judge matrix.

Primary Parameters (fixed for the main experiment)

Parameter	Default	Notes
Magnitude	5	Moderate changes per iteration
Selection mode	Deterministic	Simplest to analyse
Critic mode	images+code	Gives critic maximum information
Variants per iteration	2	Balances exploration vs API cost
Iterations per run	15	Enough to observe trajectory shape
Repeats per condition	3	Characterises stochastic variation
One-shot attempts per condition	10	Characterises the one-shot distribution

Advanced Overrides (collapsed by default)

Expandable panel allowing the researcher to sweep magnitude, selection mode, critic mode, or variants as secondary experiments. Hidden by default — the primary experiment holds them fixed.

Computed Run Count

Live display: **"This experiment defines N conditions. Per condition: 1 evolutionary run × 3 repeats + 10 one-shot baselines + cross-model evaluation = M total API sessions"**

API Keys

Fields for Anthropic, Gemini, OpenAI keys. Required for any model checked above.

---

2. Baseline Generator

For each condition (taste × model), before any evolutionary run begins, generate one-shot baseline programs.

Two baseline types

Baseline A — "From scratch": Give the model the taste description and ask it to write the best Logo turtle graphics program it can. No seed program, no iteration. Run this N times (default 10) to get a distribution.

Baseline B — "Best single edit": Give the model the seed program and say: "Improve this Logo program as much as you can in a single step to better achieve this aesthetic." This is equivalent to magnitude 10, one iteration, but framed as unconstrained improvement. Run this N times per seed.

The distinction matters: if Baseline B matches the evolutionary winner, then the value of evolution is just in the critic feedback loop, not in iteration. If the evolutionary winner exceeds both baselines, iteration itself is doing real work.

Baseline prompts

Use a dedicated system prompt (not the modifier prompt, which is tuned for constrained variation):

```
BASELINE_SYS = `You are an expert Logo turtle graphics programmer.
Write programs that create visually striking, aesthetically compelling images.
Use the full Logo command set: FD, BK, RT, LT, REPEAT, SETPC, SETBG, PU, PD, ARC,
CIRCLE, SETPW, TO...END.
Return ONLY the Logo code, no explanation.`
```

For Baseline A (from scratch):

```
Write a Logo turtle graphics program that achieves this aesthetic goal: "[taste]"
Make it as visually compelling and sophisticated as possible.
Return ONLY the Logo code.
```

For Baseline B (single edit):

```
Here is a Logo program:
[seed code]

Improve it as much as you want — rewrite completely if needed — to better achieve
this aesthetic: "[taste]"
Return ONLY the improved Logo code.
```

Baseline data model

```
Baseline {
  conditionId: string
  type: 'from_scratch' | 'single_edit'
  model: string
  taste: string
  seed: string | null          // null for from_scratch
  attemptIndex: number
  code: string
  structuralMetrics: StructuralMetrics
  imageMetrics: ImageMetrics
  criticScores: { [judgeModel: string]: CriticScore }   // filled during cross-
model evaluation
  rawPrompt: string
```

```
  rawResponse: string
  timestamp: ISO datetime
  latencyMs: number
}

CriticScore {
  aesthetic: number
  novelty: number
  total: number
  reason: string
}
```

---

3. Structural Metrics Analyser

Computed automatically for every program — baselines, evolutionary winners at each iteration, and all variants. No LLM required.

Code-level metrics (from source)

```
StructuralMetrics {
  totalCommands: number          // count of executable Logo commands
  distinctCommands: number       // unique command types used
  nestingDepth: number           // max depth of nested REPEATs
  colorChanges: number           // number of SETPC commands
  proceduresDefined: number      // number of TO...END blocks
  repeatCount: number            // number of REPEAT commands
  codeLength: number             // character count of source
  tokenCount: number             // whitespace-delimited token count
}
```

Render-level metrics (from compileLogo queue)

The `compileLogo()` function returns a queue of drawing operations. Before rendering to canvas, analyse the queue:

```
RenderMetrics {
  totalPathLength: number        // sum of all line segment lengths
  operationCount: number         // total draw operations in queue
  distinctColors: number         // unique colours in the drawing
  boundingBox: { minX, minY, maxX, maxY }
  boundingBoxCoverage: number    // fraction of canvas area covered by bounding
box
  lineCount: number              // number of line segments
  arcCount: number               // number of arc operations
}
```

Image-level metrics (from rendered canvas)

After rendering to an offscreen canvas, compute from pixel data:

```
ImageMetrics {
  distinctPixelColors: number    // unique colours in the actual rendered image
  spatialEntropy: number         // Shannon entropy of a downsampled luminance
grid (e.g. 16×16)
  fillRatio: number              // fraction of pixels that differ from background
  symmetryScore: number          // correlation between left/right halves (0-1)
  edgeDensity: number            // fraction of pixels at colour boundaries
}
```

Spatial entropy is the most informative single metric: a blank canvas has entropy 0, a random noise image has maximum entropy, and a visually interesting drawing sits somewhere in between. Compute by dividing the canvas into a 16×16 grid, averaging luminance per cell, and computing Shannon entropy of the distribution.

Implementation note: All image metrics are computed from canvas pixel data via `getImageData()`. They run in milliseconds and add no API cost.

---

4. Cross-Model Evaluator

After all evolutionary runs and baselines complete, the evaluator re-judges key programs using each evaluation model.

What gets re-judged

For each condition:

- The evolutionary winner (final iteration) from each repeat
- The best one-shot baseline (highest self-score) from each type
- Optionally: the evolutionary winner at the midpoint iteration (e.g. iteration 7 of 15)

Process

For each program × judge model pair:

1. Render the program to PNG
2. Call the judge model with a single-image critic prompt (below)
3. Record the scores

This produces a matrix: rows are programs, columns are judge models, cells are scores. If Claude-evolved programs score well when judged by Gemini, the improvement is robust.

Single-image critic prompt

```
EVAL_CRITIC_SYS = `You are an art critic evaluating a single computer-generated
image
created by a Logo turtle graphics program.

Score the image on:
- aestheticScore (1-10): visual beauty, composition, harmony relative to the
stated taste
- noveltyScore (1-10): originality, surprise, visual interest

Respond ONLY with JSON:
{"aestheticScore": N, "noveltyScore": N, "reason": "brief explanation"}`
```

Data model addition

The `Run` record gains:

```
crossModelScores: { [judgeModel: string]: CriticScore }
```

---

5. Batch Runner Engine

Data model

```
Experiment {
  id: string (timestamp-based)
  created: ISO datetime
  parameters: {
    seeds: { id: string, code: string, label: string }[],
    tastes: { id: string, text: string }[],
    models: string[],
    evalModels: string[],
    magnitude: number,
```

```
    selectionMode: string,
    criticMode: string,
    variantsPerIter: number,
    iterationsPerRun: number,
    repeatsPerCondition: number,
    oneshotAttempts: number
  }
  conditions: Condition[]
  status: 'pending' | 'running' | 'completed' | 'stopped'
}

Condition {
  id: string (e.g. "spiral_geometric_claude")
  seedId: string
  tasteId: string
  model: string
}

Run {
  conditionId: string
  repeatIndex: number
  status: 'pending' | 'running' | 'completed' | 'error'
  startTime: ISO datetime
  endTime: ISO datetime
  iterations: Iteration[]
  finalWinnerCode: string
  finalScore: number
  scoreTrajectory: number[]
  structuralTrajectory: StructuralMetrics[]
  imageTrajectory: ImageMetrics[]
  crossModelScores: { [judgeModel: string]: CriticScore }
  error: string | null
}

Iteration {
  index: number
  baseCode: string
  variants: VariantRecord[]
  winnerCode: string
  winnerScore: number
  winnerAestheticScore: number
  winnerNoveltyScore: number
  feedback: string
  selectionLog: string
  structuralMetrics: StructuralMetrics
  renderMetrics: RenderMetrics
  imageMetrics: ImageMetrics
```

```
  rawModifierPrompt: string
  rawModifierResponse: string
  rawCriticPrompt: string
  rawCriticResponse: string
  timestampStart: ISO datetime
  timestampEnd: ISO datetime
  modelUsed: string
  latencyMs: { modifier: number, critic: number }
}

VariantRecord {
  code: string
  aestheticScore: number
  noveltyScore: number
  totalScore: number
  reason: string
  isWinner: boolean
  structuralMetrics: StructuralMetrics
}
```

Execution order

```
Phase 1: Baselines
  for each condition (taste × model):
    generate N from-scratch baselines
    for each seed in this condition:
      generate N single-edit baselines

Phase 2: Evolutionary runs
  for each condition (seed × taste × model):
    for repeat = 1 to repeatsPerCondition:
      run evolution for iterationsPerRun iterations
      compute metrics at each iteration

Phase 3: Cross-model evaluation
  for each completed run's final winner:
    for each evalModel:
      score the winner with evalModel
  for each baseline's best program:
    for each evalModel:
      score with evalModel
```

Phases run sequentially. The user can stop between phases. Partial results are retained and exportable.

Key refactoring of `runStep()`

The existing `runStep()` reads parameters from the DOM and writes results to the DOM. The experiment version needs a `runStepPure(params)` that:

- Takes `{baseCode, feedback, numVariants, magnitude, taste, model, selectionMode, criticMode}` as explicit arguments
- Returns `{iteration: Iteration, winnerCode, feedback}` with all data including raw API captures
- Does NOT touch the DOM
- The batch runner calls this in a loop, then updates the progress UI separately

Forcing the model

For experiments, when a specific model is being tested, fallback should be disabled. Add a parameter `forceModel` to `callAI` that bypasses fallback and throws on failure, so the run is marked as errored rather than silently switching models. The `modelUsed` field in each iteration records what actually happened.

Progress UI

A card showing:

- Current phase: "Phase 1: Generating baselines" / "Phase 2: Evolution" / "Phase 3: Cross-model evaluation"
- Overall progress bar: "Run 14 of 90 (15.6%)"
- Current condition: "seed: spiral, taste: geometric, model: Claude"
- Current iteration: "Iteration 3 of 15"
- Elapsed time and rough ETA
- **Stop** button (finishes current iteration, then stops — partial data retained)
- Live mini-canvas showing the current program being evolved

---

6. Raw API Logger

Every `callAI()` invocation during an experiment records:

```
APICall {
  runId: string | null
  baselineId: string | null
  iterationIndex: number | null
  agent: 'modifier' | 'critic' | 'baseline_generator' | 'cross_model_evaluator'
  modelRequested: string
  modelActuallyUsed: string
  systemPrompt: string
  userPrompt: string
  imageCount: number
  responseText: string
  latencyMs: number
  timestamp: ISO datetime
  httpStatus: number
  error: string | null
}
```

Implementation: modify `callAI` in the experiment version to return `{text, meta}` where `meta` contains the above fields.

---

7. Results Dashboard

7a. The Key Chart: Evolution vs Baseline

The primary visualisation answering the research question.

For each condition (seed × taste × model), show:

- The distribution of one-shot baseline scores as a dot strip or box plot
- The evolutionary winners' final scores as highlighted markers
- The evolutionary score trajectory as a small sparkline beside it

This makes it immediately visible whether evolution exceeds the one-shot ceiling.

Aggregate summary: Across all conditions, what fraction of evolutionary runs exceeded the best one-shot baseline for the same condition? By how much on average?

7b. Score Trajectory Chart

Line chart of score vs iteration, one line per run, colourable/groupable by model, taste, or seed. With the one-shot baseline score range shown as a horizontal band.

7c. Structural Metrics Trajectory

Line charts of key structural metrics (distinctCommands, nestingDepth, colorChanges, codeLength) vs iteration. Answers: does evolution increase program complexity over time?

7d. Cross-Model Agreement Matrix

Heatmap: rows = programs (grouped as "evolved" vs "baseline"), columns = judge models. Cell colour = score.

7e. Run Table

Sortable table of all completed runs:

Seed	Taste	Model	Repeat	Final Score	vs Best Baseline	Trajectory	Complexity	Time

- **vs Best Baseline**: difference between final score and best one-shot score for same condition. Green if positive.
- **Trajectory**: sparkline (inline SVG polyline, 60×20)
- Clicking a row expands to show iteration detail

7f. Visual Gallery

Grid of final winner canvases paired with best one-shot baseline for comparison, labelled with condition parameters. Evolved winner on left, best baseline on right. Clicking opens modal with code + playback.

---

8. Export System

JSON export (primary)

The entire `Experiment` object as a single JSON file including all conditions, runs, iterations, baselines, API call metadata, and all metrics.

Filename: `experiment_{id}_{timestamp}.json`

CSV exports

runs.csv — one row per evolutionary run:

```
condition_id, seed_label, taste, model, repeat_index, iterations_completed,
final_score, final_aesthetic, final_novelty,
best_baseline_a_score, best_baseline_b_score,
score_vs_baseline_a, score_vs_baseline_b,
final_distinct_commands, final_nesting_depth, final_code_length,
final_spatial_entropy, final_fill_ratio,
elapsed_ms, error
```

iterations.csv — one row per iteration across all runs:

```
condition_id, repeat_index, iteration,
winner_score, winner_aesthetic, winner_novelty,
distinct_commands, nesting_depth, color_changes, code_length,
spatial_entropy, fill_ratio, symmetry_score,
num_variants_generated, num_variants_valid,
model_used, modifier_latency_ms, critic_latency_ms
```

baselines.csv — one row per baseline attempt:

```
condition_id, type, model, taste, seed_label, attempt_index,
score_aesthetic, score_novelty, score_total,
distinct_commands, nesting_depth, code_length,
spatial_entropy, fill_ratio
```

cross_model_scores.csv — one row per program × judge combination:

```
program_type, condition_id, repeat_index, judge_model,
aesthetic_score, novelty_score, total_score, reason
```

PNG gallery export

A zip file containing:

- `evolved/{condition_id}_rep{n}.png` — final evolutionary winners
- `baseline_scratch/{condition_id}_attempt{n}.png` — from-scratch baselines
- `baseline_edit/{condition_id}_seed{s}_attempt{n}.png` — single-edit baselines

---

9. UI Structure

Two tabs at the top:

Tab 1: Setup & Run

- Experiment Definition Panel (section 1)
- Batch Runner progress (section 5) — appears when running
- Live mini-canvas

Tab 2: Results

- Evolution vs Baseline chart (7a) — the headline finding
- Score Trajectory chart (7b)
- Structural Metrics chart (7c)

- Cross-Model Agreement matrix (7d)
- Run Table (7e)
- Visual Gallery (7f)
- Export buttons (section 8)

Header: Logo Evolution Lab branding + chip `EXPERIMENT EDITION`.

---

10. State Management

```javascript
const EXP = {
  experiment: null,
  runs: [],
  baselines: [],
  apiLog: [],
  currentPhase: null,      // 'baselines' | 'evolution' | 'cross_eval'
  currentRunIndex: -1,
  isRunning: false,
  stopRequested: false,
};
```

Data held in memory. Export buttons serialise from this object. `beforeunload` warning if unsaved data exists.

---

11. Implementation Notes for Sonnet

Build order (phased)

Phase A — Core engine (build first, test immediately)

1. Copy existing HTML file as starting point
2. Strip interactive UI HTML but keep ALL JavaScript functions
3. Refactor `runStep()` → `runStepPure(params)` (explicit params, no DOM, returns structured data)
4. Modify `callAI()` to return `{text, meta}` with raw prompt/response and `forceModel` option
5. Implement `computeStructuralMetrics(code)` and `computeImageMetrics(canvas)`
6. Build the Baseline Generator (two prompt types + collection logic)
7. Build the Batch Runner loop (all three phases)
8. Build minimal progress UI

9. Build JSON export

Phase B — Results dashboard 10. Run Table with sparklines 11. Evolution vs Baseline chart (the key visualisation) 12. CSV exports 13. Visual Gallery with paired comparisons

Phase C — Polish 14. Score Trajectory chart 15. Structural Metrics trajectory chart 16. Cross-Model Agreement heatmap 17. Advanced parameter overrides panel 18. PNG zip export

Key refactoring details

`runStepPure(params)` must:

- Accept `{baseCode, feedback, numVariants, magnitude, taste, model, selectionMode, criticMode}`
- Call modified `agentModify` / `agentCritique` that take explicit arguments instead of reading DOM
- Return full `Iteration` record including raw API data and computed metrics
- Never touch the DOM

`computeStructuralMetrics(code)`:

- Parse Logo source to count commands, measure nesting, etc.
- Can reuse parts of `compileLogo`'s tokenizer
- Pure function, no side effects

`computeImageMetrics(canvas)`:

- Takes a canvas element with program already rendered
- Uses `getImageData()` to access pixels
- Returns `ImageMetrics` object
- Pure function

Sparklines

Inline SVG polylines in a 60×20 viewBox:

```javascript
```

```
function sparkline(values) {
  const max = Math.max(...values), min = Math.min(...values);
  const points = values.map((v, i) =>
    `${(i / (values.length - 1)) * 60},${20 - ((v - min) / (max - min || 1)) * 18}`
  ).join(' ');
  return `<svg viewBox="0 0 60 20" width="60" height="20"><polyline points="${points
}
```

Charts

Hand-drawn inline SVG for all charts. Data volumes are small. No charting library needed.

File size

Will be larger than the 160KB original. That's fine for a researcher's tool. Single self-contained HTML file.

---

12. Research Workflows This Enables

Primary experiment: Does evolution beat one-shot?

Setup: 3 seeds × 3 tastes × 3 models, 15 iterations, 3 repeats, 10 one-shot baselines per condition.
Total: 27 evolutionary runs + 270 baselines.
The evolution-vs-baseline chart directly answers the question.

Secondary experiments (via advanced overrides)

1. **Magnitude sweep**: Fix one condition, sweep magnitude 1–10, compare trajectories.
2. **Selection mode comparison**: Deterministic vs probabilistic across many runs.
3. **Critic mode comparison**: images+code vs images-only — compare structural metrics of winners.
4. **Seed complexity effect**: Simple seeds vs complex seeds — do they converge to similar quality?
5. **Human-AI agreement**: Export visual gallery, show to human raters, compare with critic scores.

---

13. What This Document Does NOT Cover

- Statistical analysis code (researcher uses Python/R on exported CSVs)
- Headless/CLI execution (remains a browser app)
- Multi-user or server-side features (single-user, client-side only)

- Changes to the original Logo Evolution Lab app
- The human evaluation instrument (separate tool consuming PNG exports)